

Autoencoders (AEs) and Variational Autoencoders (VAEs)

Unsupervised Representation Learning Models

Previous session

Types of Autoencoders: Architecture and Code-Level Understanding

- Shallow Autoencoder
- Deep Autoencoder
- CNN Autoencoder
- Denoising Autoencoder

Today's Session Overview

Limitations of Autoencoders

Variational Autoencoder:

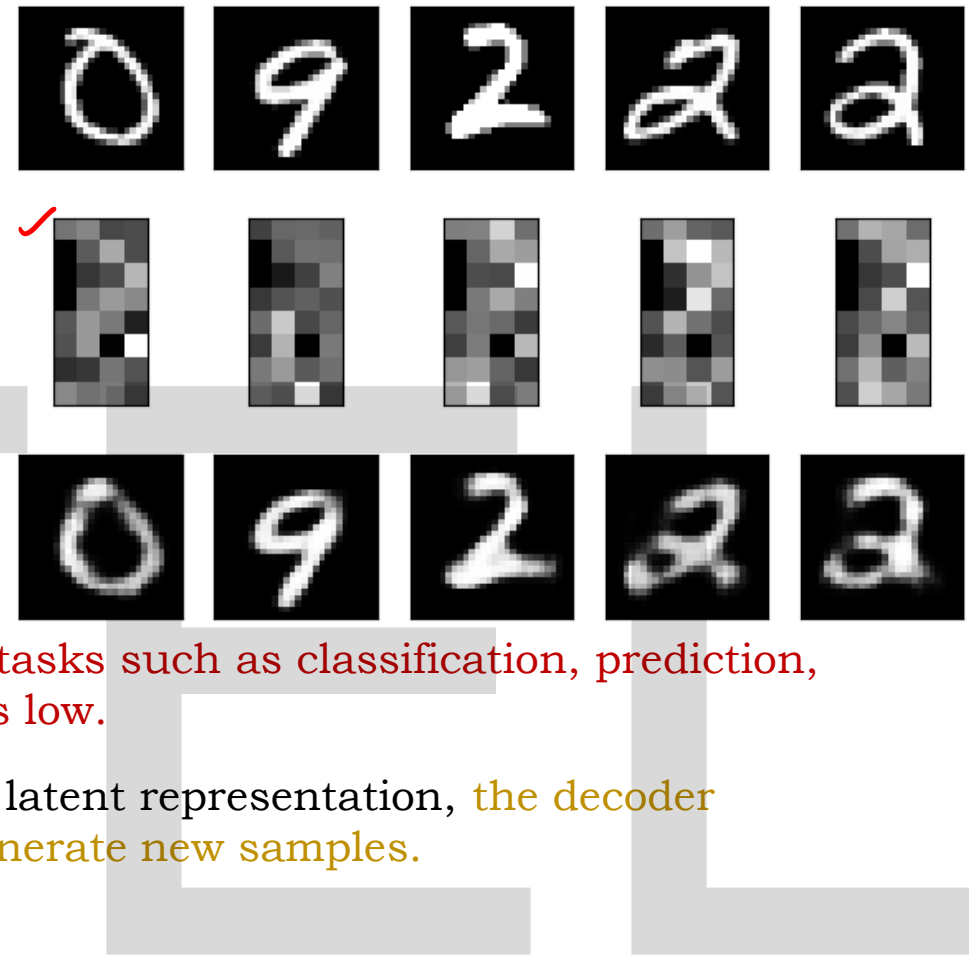
- Architecture
- Loss function

NPTEL

Limitations of Autoencoders

Since the latent space contains a compressed representation of the input, it is optimized only for reconstruction.

1. There is **no guarantee that it captures meaningful factors**. An autoencoder may still reconstruct the input with low reconstruction error, but the learned latent features **may not be semantically meaningful features**—i.e., features that cannot be easily understandable by the humans or task-relevant features such as digit identity, shape, or stroke thickness.
2. This compressed information is **not guaranteed to be useful for tasks such as classification, prediction, clustering, or anomaly detection, even if the reconstruction error is low**.
3. Since the reconstructed output is generated from a compressed latent representation, **the decoder produces only an approximation of the input, the model cannot generate new samples**.
4. Sensitivity to Latent Space Size
 - (a) **Latent space too large (with no regularization)**: This results in overfitting and leads to trivial reconstruction, where the model reconstructs inputs well but fails to learn any useful or compact representation.
 - (b) **Latent space too small**: Then the model is forced to compress the input excessively, leading to information loss, producing poor reconstruction quality.



Variational Autoencoders (VAEs) address these issues by introducing **regularization on the latent space**.

This is achieved by adding a **KL divergence regularization term** to the **loss function**.

Variational Autoencoders (VAEs)

Given a dataset of N samples:

$$X = \{x^{(1)}, x^{(2)}, \dots, x^{(N)}\}$$

Each class follow its own data distribution

$\{c_1, c_2, c_3\}$ $\{x_{15}, x_{16}, \dots\}$

$\{x_1, x_2, x_3, x_4, \dots\}$ $\{x_8, x_9, x_{10}, x_{11}, \dots\}$

$p(x | c_1)$ $p(x | c_2)$ $p(x | c_3)$

- We do not know the true data distribution $p_{data}(x)$,
- we do not know its exact functional form (Gaussian, Bernoulli, Binomial, Poisson, etc.) nor its parameters.
- We only have samples (dataset) drawn from this unknown distribution.

sepallength	sepalwidth	petallength	petalwidth	class
5.1	3.5	1.4	0.2	Iris-setosa
4.9	3.0	1.4	0.2	Iris-setosa
4.7	3.2	1.3	0.2	Iris-setosa
4.6	3.1	1.5	0.2	Iris-setosa
5.0	3.6	1.4	0.2	Iris-setosa
...	...	...	...	...
6.7	3.0	5.2	2.3	Iris-virginica
6.3	2.5	5.0	1.9	Iris-virginica
6.5	3.0	5.2	2.0	Iris-virginica
6.2	3.4	5.4	2.3	Iris-virginica
5.9	3.0	5.1	1.8	Iris-virginica

Goal (Generative Modelling)

The goal of the generative model is to learn this unknown data distribution and to **generate new samples** x_{new} that look like the training data, i.e., samples drawn from the true data distribution $p_{data}(x)$:

$$x_{new} \sim p_{data}(x)$$

Variational Autoencoders (VAEs)

Two distributions

1. Data space distribution:

$$p_{data}(x)$$

This represents the (unknown) distribution of the given data samples.

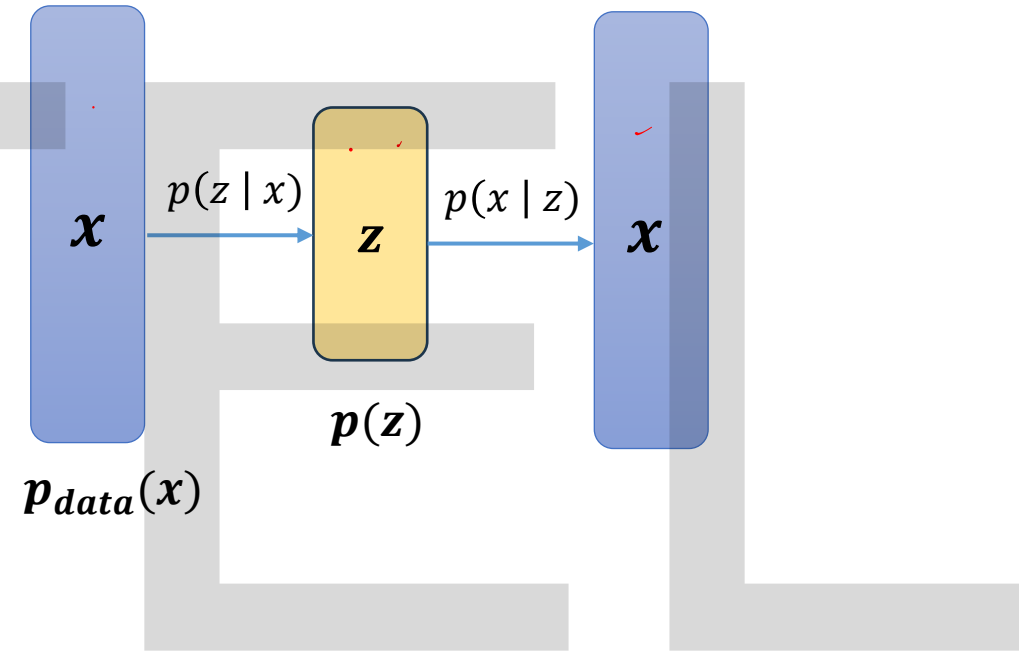
$$x_1, x_2, x_3, \dots, x_4$$
$$p_{data}(x)$$

2. Latent space distribution:

$$p(z)$$

x and z live in different spaces means they have different dimensionality and structure

Therefore, we need probabilistic mappings to connect:
data space $\leftrightarrow$ latent space



Two key probabilistic mappings in VAE

1. Posterior distribution (Encoder)

$$p(z | x)$$

- Maps an input data point x to a latent representation z
- Interpreted as:

The probability distribution of the latent variable z given the data sample x

2. Likelihood distribution (Decoder)

$$p(x | z)$$

- Maps latent variable z back to data space
- Interpreted as:
The probability of reconstructing data x given the latent z

Variational Autoencoders (VAEs): Encoder

The primary objective is to infer the **posterior distribution**:
 $p(z | x)$

The posterior $p(z | x)$ depends on:

$$p(z | x) = \frac{p(x | z) \cdot p(z)}{p(x)}, \text{ from Bayes rule}$$

In a VAE, the latent variable is not a single number, but a vector:

$$z = [z_1, z_2, \dots, z_d]$$

Each z_i is called a latent dimension (or latent variable).

Suppose if the input data has 100 features, then
 $100 \rightarrow \text{Encoder} \rightarrow 25$.

$\therefore$ The latent variable z is not a single number, but a 25-dimensional vector.

The primary objective is to learn this latent space. We do not know the structure of the latent space. i.e. **the true latent distribution $p(z)$ is unknown**.

$\therefore$ we assume that $p(z)$ is a standard normal distribution with mean 0 and variance 1.

$$p(z_i) = \mathcal{N}(0, 1) \rightarrow \text{Prior probability}$$

This enforces a well-structured and continuous latent space.

$$p(x) = \int p(x | z) \cdot p(z) dz$$

Computing $p(x)$ is very hard. But why?

Lets, come back to this in a while.

$\therefore$ We do not directly compute $p(z | x)$; instead, we choose $q(z | x)$, a variational approximation to the true posterior for computation.

$q(z | x)$ to be Gaussian is an assumption.

$$p(z | x) \approx q(z | x) = \mathcal{N}(\mu(x), \sigma^2(x))$$

The parameters $\mu(x)$ and $\sigma(x)$ of the approximate posterior distribution $q(z | x)$ are to be learned by the encoder network.

Why we assumed $q(z | x)$ to be gaussian?

input (x) $\xrightarrow{\text{passed}}$ Encoder $\xrightarrow{\text{learns}}$ $q(z | x) [\mu(x) \text{ and } \sigma^2(x)]$

The prior $p(z)$ serves as a reference distribution for $q(z | x)$. $\therefore$ from input only we get the latent space.

$p(z_i) = \mathcal{N}(0, 1)$ Who ensures?

KL divergence term ensures that the learned encoder distribution remains close to this prior, preventing the latent space from becoming irregular or unstructured.

Variational Autoencoders (VAEs): Decoder

Once $\mu(x)$ and $\sigma(x)$ of $q(z | x)$ are learned by the encoder then

$$z \sim q(z | x) = \mathcal{N}(\mu(x), \sigma^2(x))$$

z is randomly drawn. Once we have the latent variable z , the next is **decoder distribution**: $p(x | z)$
What type of data do we want the decoder to reconstruct / generate?

The decoder is a neural network that takes z as input and outputs distribution parameters.

Output distribution parameters depends on type of data:

Continuous data (Gaussian likelihood)

$$p(x | z) = \mathcal{N}(\mu(z), \sigma^2)$$

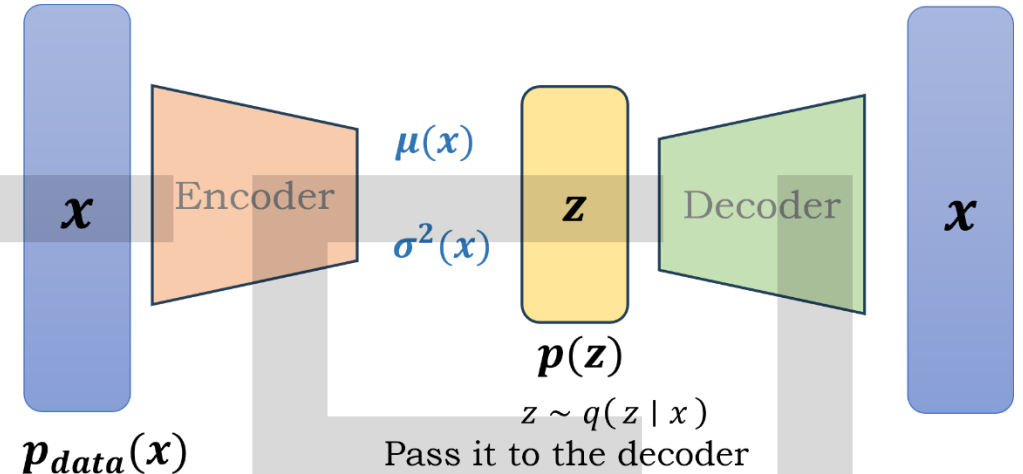
- Decoder outputs the parameters of a gaussian: $\mu(z)$
- σ^2 is often fixed
- Loss is MSE

Binary data (Bernoulli likelihood)

$$p(x | z) = \text{Bernoulli}(\pi(z))$$

- Decoder outputs: $\pi(z)$ (probabilities)
- Loss is BCE

Then x is sampled from the distribution defined by the decoder



The decoder does two things:

1. **Outputs the parameters** of the distribution $p(x | z)$
2. **Generates x** by sampling from that distribution

The goal is:

Reconstructed data x should be as close as possible to the original input x

This closeness is measured by the **log-likelihood term**:

$$\mathbb{E}_{q(z|x)}[\log p(x | z)]$$

Variational Autoencoders (VAEs): Decoder

Why computing $p(x)$ is very hard

In the forward generative process, a latent variable z is first sampled, and then the data x is generated by passing z to the decoder through $p(x | z)$.

Once the mean μ and variance σ^2 are known from $q(z | x)$,
and **since** $z \sim q(z | x)$ there exist infinitely many possible values of z that can be sampled from the Gaussian distribution defined by that mean and variance.

Suppose say the reconstructed x is $[5.1, 3.5, 1.4, 0.2]$.

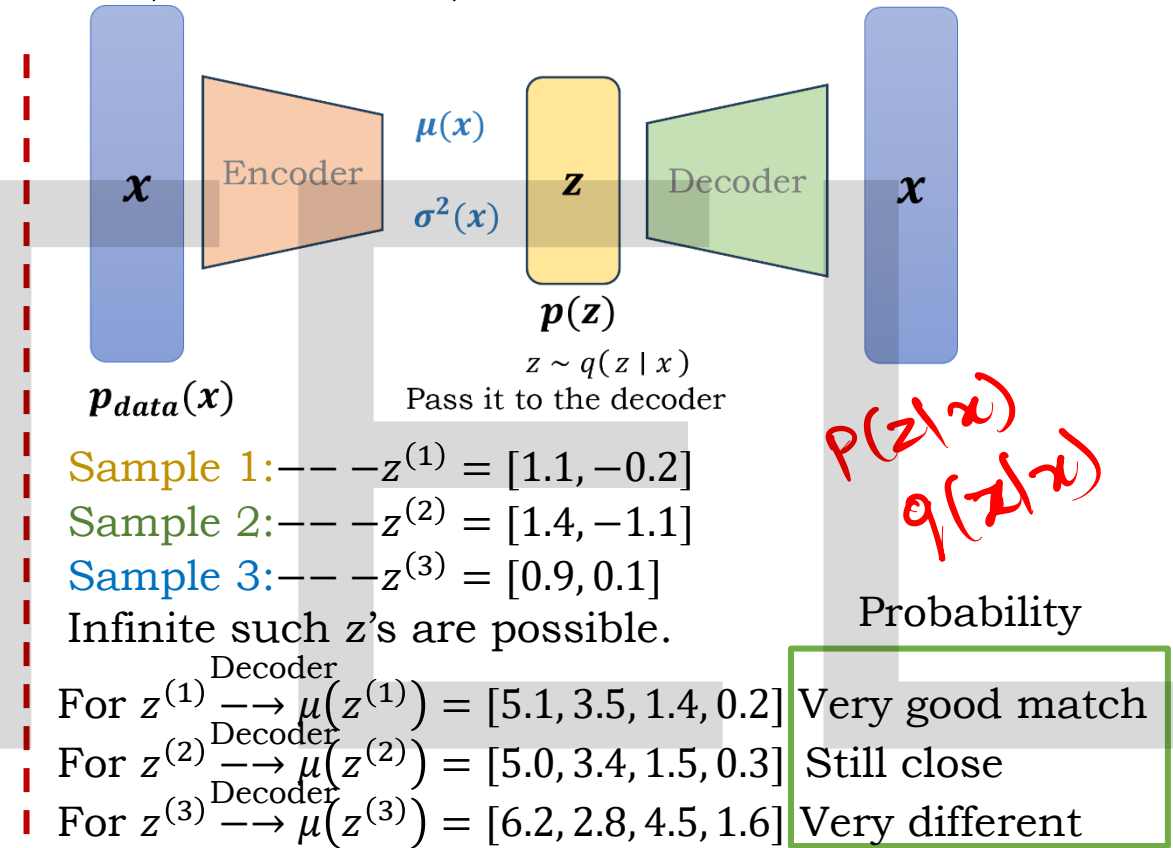
Latent dimension (z) = 2

Encoder outputs:

$$\begin{aligned}\mu(x) &= [\mu_1(x), \mu_2(x)] \rightarrow \mu(x) = [1.2, -0.5] \\ \sigma^2(x) &= [\sigma_1^2(x), \sigma_2^2(x)] \rightarrow \sigma(x) = [0.3, 0.7]\end{aligned}$$

This defines the **distribution**:

$$q(z | x) = \mathcal{N}([1.2, -0.5], [0.3^2, 0.7^2])$$



Which z produced this x ? $p(x | z)$

$$p(x) = \int p(x | z) \cdot p(z) dz$$

The probability of observing x is obtained by summing the probability contributions from **all possible latent variables** weighted by how likely they are.

Variational Autoencoders (VAEs): Loss Function

A VAE does two things:

1. Reconstruct the input well (like a normal autoencoder)
2. Force the latent space to follow a known distribution (so we can *generate* new data)

That is why the loss has two terms:

$$\mathcal{L}(x) = \underbrace{-\mathbb{E}_{q(z|x)}[\log p(x|z)]}_{\text{Reconstruction / Data consistency}} + \underbrace{KL(q(z|x) \parallel p(z))}_{\text{Regularization}}$$

Reconstruction / Data consistency Regularization

Mean square error

Loss function depends on a random variable z .
 z is sampled from a distribution:

$$z \sim q(z|x) = \mathcal{N}(\mu(x), \sigma(x)^2)$$

- Sampling is a random operation ✓
- Random sampling breaks gradient flow ↙
- Neural networks are trained using gradient descent
- Backpropagation cannot pass through randomness

In VAE, we only know the encoder distribution $q(z|x)$ and a simple distribution $p(z)$ as a reference.

The KL term forces the encoder's latent space to match this reference distribution.

$$L = L_{KL}(\mathcal{N}(\mu, \sigma^2) \parallel \mathcal{N}(0, 1))$$

$$L_{KL} = \frac{1}{2} [\mu^2 + \sigma^2 - \log \sigma^2 - 1]$$

For the 2 gaussian distribution, the KL divergence will take this form

So, the question is: **How do we compute gradients when part of the network involves sampling?**

This motivates the Reparameterization Trick ✓

Next Session

➤ Reparameterization Trick

NPTREL

NIPTEEL

Thank you